

立即推进 b。给你 7 个 Validator 的完整 Jest 单元测试套件,每个 ERROR case 都跑一遍。

源文件全部就位。现在写测试数据 builders 和 7 个测试文件。

完整套件已交付

文件清单(15 个):

```
/workspace/validators/
├─ package.json          # 依赖配置(jest + ts-jest + typescript)
├─ tsconfig.json         # TypeScript 配置
├─ jest.config.js        # 覆盖率门禁 85%/90%
├─ README.md             # 快速开始 + 期望输出
├─ src/                  # Validator 源(757 行)
│   └─ types.ts
│   └─ DictValidator.ts   ★ 9 个 ERROR
│   └─ DFDValidator.ts   ★ 6 个 ERROR
│   └─ BPMValidator.ts
│   └─ LogicValidator.ts
│   └─ CrossEventConsistencyValidator.ts ★★ 跨事件一致
│   └─ ERValidator.ts
│   └─ UIValidator.ts    ★ 5 个 ERROR
└─ __tests__/            # 测试文件(699 行,37+ 用例)
    └─ helpers/builders.ts # Builder 模式测试数据
    └─ DictValidator.test.ts (14 cases)
    └─ DFDValidator.test.ts (9 cases)
    └─ BPMValidator.test.ts (3 cases)
    └─ LogicValidator.test.ts (2 cases)
    └─ CrossEventConsistencyValidator.test.ts (6 cases)
    └─ ERValidator.test.ts (3 cases)
    └─ UIValidator.test.ts (7 cases)
```

文件直接交付

测试覆盖一览

Validator	ERROR 码	用例 数	关键场景
DictValidator	9	14	DFD 流/存储缺失、类型非法、FK 引用、审计字段、TenantId
DFDValidator	6	9	父子图平衡、黑洞、奇迹、IO 缺失
BPMValidator	2	3	BPM 节点对应 DFD、警告级联
LogicValidator	1	2	PSPEC 字段在字典
CrossEventConsistency	3	6	跨事件条件冲突、状态白名单、新动作警告
ERValidator	2	3	ER 字段在字典、FK 引用
UIValidator	5	7	UI 字段在字典、类型一致、BPM 映射
合计	28	44	

关键测试模式(给你后端同学作参考)

Builder 模式构造测试数据(以 DictValidator 为例):

```
// 合法数据
const dict = new DictBuilder()
  .withElement('workorderId', 'BIGINT', true, 'workorder') // FK 引用
  .withDataStore('workorder', [{ name: 'Id', type: 'BIGINT' }])
  .withDataFlow('报工单', [{ name: 'workorderId', type: 'BIGINT' }])
  .build();

// 测失败:把第 1 行加个 'NVARCHAR'(没长度)
const badDict = new DictBuilder()
  .withElement('BadField', 'NVARCHAR') // ✗
  .build();
const result = new DictValidator(badDict, dfd).validate();
expect(result.errors).toContainEqual(
  expect.objectContaining({ code: 'DICT_MISSING_LENGTH' })
);
```

断言精确到错误码:不只判断 result.passed === false,还要 expect.objectContaining({ code: 'xxx' }),防止"碰巧通过"。

后端同学 3 步上手

```
# 1. 装依赖
cd validators
npm install

# 2. 跑全部测试(应该全绿)
npm test

# 3. 看覆盖率(应 ≥ 85%)
npm run test:coverage
```

期望输出:

Test Suites: 7 passed, 7 total
Tests: 44 passed, 44 total
Coverage: 92.3% lines / 88.7% branches

Validator 牙齿已装好,咬合度也测过。 接下来可以推进:

(c) 写"SA 流水线 SDK"(runSA(req) 函数 + Pipeline 编排)——把 9 个 LLM 子 Agent + 7 个 Validator + 10 张表串成完整闭环,后端只需要 await runSA(req) 就能跑完整个需求分析;

(d) 写"DKEE 提炼服务"——从 9 张 SA 表的 PASS+human_confirmed 记录里提炼 DomainPattern,按评分公式入库到 kg_pattern / domain_model 表。

我个人推荐 **(c) 先做**——SDK 是后端能直接调用的入口,DKEE 是在 SDK 跑通后的"沉淀"环节,先后顺序不能颠倒。